

Skill Definition & Analysis Report

AI Billing Proxy Infrastructure
9 Skills — 11 Analysis Dimensions Each

Cross-Skill Compatibility, Stackability & Integration Mapping

Generated: 2026-06-02

Table of Contents

1. Executive Summary
2. Skill Inventory & Quick Reference
3. Purpose & Individual Use-Case Comparison
4. Misconception & Conspiracy Analysis
5. Mostly Overlooked Features
6. Contradictions & Resolutions
7. Compatibility Matrix (Compatible Types)
8. Incompatibility Matrix (Not Compatible Types)
9. Stackability Analysis
10. Best Combination Synergies
11. Worst Combination Conflicts
12. Integration Architecture Diagrams
13. File Inventory & Local Storage Map

1. Executive Summary

This report defines and analyzes nine critical skills for an AI billing proxy infrastructure. Each skill has been profiled across eleven analysis dimensions: purpose, individual use-case, misconception, conspiracy, mostly overlooked features, contradictions, compatible types, incompatible types, stackability, stackable best combined to, and stackable worst combined to. The nine skills span three functional layers: the client layer (Opencode Owl Install Proxy, Openhuman Owl Install Proxy), the service layer (Combined Proxy Billing, API Gateway Skill, OpenRelay Go), the infrastructure layer (Deployment Manager, Persistent Memory), and the integration layer (Browser-Use, MCP Builder). Together, they form a complete stack for managing AI API billing, routing, and observability across multiple providers and clients.

The analysis reveals three key findings. First, Persistent Memory is the most universally compatible skill — every other skill benefits from state persistence, making it the foundation of the stack. Second, Combined Proxy Billing and API Gateway Skill form the core service duo: billing tracks costs, gateway routes requests, and together they provide full-stack AI API management. Third, the owl install proxies (Opencode and Openhuman) are the entry points that connect end-user tools to the infrastructure, while Browser-Use and MCP Builder serve as glue skills that automate workflows and expose capabilities to AI agents.

2. Skill Inventory & Quick Reference

Skill	Layer	Language	Port	Key Function
Opencode Owl Install	Client	Bash	N/A	Auto-config OpenCode CLI for billing proxy
Openhuman Owl Install	Client	Bash	8400	Auto-config OpenHuman runtime for billing proxy + OAuth2 PKCE
Combined Proxy Billing	Service	Python	8090	Multi-provider billing aggregation + quota enforcement
OpenRelay Go	Service	Go	8443	Network-level AI API relay with auto-discovery of 45+ endpoints
API Gateway Skill	Service	Python	8000	Application-layer smart routing + rate limiting + caching
Browser-Use	Integration	JS/Node	N/A	Natural-language browser automation for OAuth flows
Deployment Manager	Infrastructure	YAML/Bash	N/A	Full-lifecycle deployment orchestration + canary + rollback
Persistent Memory	Infrastructure	Python	6380	Cross-session state store with TTL + OCC + snapshots
MCP Builder	Integration	TypeScript	stdio	MCP server construction for AI agent tool exposure

3. Purpose & Individual Use-Case Comparison

Skill	Purpose (1-line)	Primary Use-Case
Opencode Owl Install	Zero-friction client-side proxy configuration for OpenCode CLI	Solo dev or team routes all OpenCode AI calls through a shared billing proxy in under 60 seconds
Openhuman Owl Install	Client-side proxy install with OAuth2 PKCE for human-AI workflows	Product team attributes AI costs per-human-reviewer across OpenHuman moderation workflows
Combined Proxy Billing	Centralized multi-provider billing aggregation and cross-provider quota enforcement	Startup with 3 AI providers sees unified spend dashboard with per-team quota alerts
OpenRelay Go	Network-level AI API relay with auto-discovery of 45+ AI endpoints	Agency with 20 devs using different AI tools tracks spending via single network proxy
API Gateway Skill	Smart request routing, rate limiting, and provider selection middleware	SaaS routes French requests to Claude, English to GPT-4o via single gateway endpoint

Skill	Purpose (1-line)	Primary Use-Case
Browser-Use	Natural-language browser automation for interactive web flows	DevOps automates OAuth2 consent flow across 15 developer machines in under 2 min each
Deployment Manager	Full-lifecycle deployment orchestration with canary and rollback	Platform team deploys billing service to staging, smoke tests, promotes to production automatically
Persistent Memory	Cross-session state persistence with TTL and optimistic concurrency	Token refresh watchdog survives restarts; quota state persists across billing service restarts
MCP Builder	MCP server toolkit to expose tools/resources/prompts to AI agents	Claude Code queries spending, checks quotas, and suggests optimizations via MCP tools

4. Misconception & Conspiracy Analysis

Skill	Top Misconception	Conspiracy Theory
Opencode Owl	It is a VPN/network tunnel capturing all outbound traffic	It is a MITM attack by design that logs all prompts and completions
Openhuman Owl	It is for humans making API calls directly; it is just a renamed Opencode owl	Shadow mode is a built-in data exfiltration channel; per-reviewer billing is surveillance
Combined Billing	It replaces individual proxy billing systems	It enables organizations to underreport AI usage to providers via billing manipulation
OpenRelay Go	It is a crack/keygen providing free AI API access	Auto-discovery is a network surveillance tool monitoring 45+ endpoints for employee activity
API Gateway	It is a load balancer distributing requests across providers for the same model	Cost optimization routes to cheapest provider while charging premium prices
Browser-Use	It is a web scraper; it requires a headed browser	It can automate phishing attacks to capture credentials
Deployment Mgr	It is only for frontend deployment (Vercel/Netlify)	Automatic rollback is a mechanism to secretly revert security patches
Persistent Memory	It is just Redis or just SQLite	It records everything an AI agent does, creating permanent surveillance records
MCP Builder	It creates AI chatbots; it only works with Claude	MCP is Anthropic's mechanism to control the AI agent ecosystem

5. Mostly Overlooked Features

Skill	Overlooked Feature 1	Overlooked Feature 2
Opencode Owl	Quota pre-flight check: fails fast on exhausted quota, saving round-trips	Automatic retry with exponential backoff on upstream 429 responses
Openhuman Owl	Zero-downtime token refresh: auto re-auth without interrupting in-flight requests	Billing tags: arbitrary KV pairs for granular cost attribution beyond team/workflow level
Combined Billing	Cost anomaly detection: alerts on spending spikes using exponential moving average	Budget forecasting: projects end-of-month spend based on historical patterns
OpenRelay Go	Provider failover chains: transparent fallback to alternate provider when primary is down	Connection warm-up: pre-establishes TLS connections at startup for 200-500ms latency reduction
API Gateway	Semantic caching: serves cached responses for semantically similar prompts using embeddings	Request priority queue: critical requests jump to front when at capacity
Browser-Use	Session checkpoint and resume: resumes from last successful step on failure	Visual assertion: screenshot comparison for visual regression detection
Deployment Mgr	Canary deployments: routes 5% traffic to new version, auto-promotes or rolls back	Deployment dry-run: validates configuration without actually deploying
Persistent Memory	Optimistic concurrency control: prevents race conditions on concurrent key updates	State snapshots: point-in-time capture for restore after failed deployments
MCP Builder	Tool annotations: readOnlyHint/destructiveHint signal AI agents to confirm before dangerous ops	Evaluation mode: auto-validates MCP servers against the specification

6. Contradictions & Resolutions

Skill	Core Contradiction	Resolution
Opencode Owl	Transparency (invisible to user) vs. Control (must surface quota denials)	Fail-open mode: seamless when working, informative when broken; bypass logging for audit
Openhuman Owl	Auditability (detailed trails) vs. Privacy (recording human identity and actions)	Configurable audit granularity: full bodies / metadata only / aggregate stats only
Combined Billing	Real-time enforcement vs. eventual consistency (events arrive with different latencies)	Soft limit + hard limit model: soft triggers alerts, hard triggers immediate block
OpenRelay Go	Transparency (invisible) vs. Control (admin can route/throttle/block without consent)	Configurable observability: audit / billing / passthrough modes
API Gateway	Single-endpoint simplicity vs. multi-provider routing complexity	Routing policy simulator: test rules against sample requests before production deploy
Browser-Use	Automation (automate as much as possible) vs. Security (credential entry risk)	Human-in-the-loop for credentials: automates everything except password entry
Deployment Mgr	Automation speed vs. Safety (fastest deploy skips all checks)	Configurable speeds: express (minimal) / standard (auto-approve) / careful (human approval)
Persistent Memory	Durability (fsync on every write) vs. Performance (async writes)	Configurable durability: sync / async / memory levels
MCP Builder	Standardization (MCP schema limits) vs. Flexibility (unique tool patterns)	Resource subscriptions + prompt templates as extension points

7. Compatibility Matrix (Compatible Types)

Skill	Match 1	Match 2	Match 3	Match 4	Match 5	Match 6	Match 7
Opencode Owl	api-gateway HIGH	persistent-mem HIGH	deploy-mgr HIGH	combined-bill HIGH	mcp-builder MED	browser-use MED	openrelay MED
Openhuman Owl	opencode-owl HIGH	combined-bill HIGH	persistent-mem HIGH	browser-use HIGH	api-gateway MED	deploy-mgr MED	mcp-builder MED
Combined Billing	opencode-owl HIGH	openhuman-owl HIGH	openrelay HIGH	api-gateway HIGH	persistent-mem MED	deploy-mgr MED	mcp-builder MED
OpenRelay Go	combined-bill HIGH	api-gateway HIGH	deploy-mgr HIGH	persistent-mem MED	opencode-owl MED	openhuman-owl MED	browser-use LOW
API Gateway	combined-bill HIGH	openrelay HIGH	opencode-owl HIGH	openhuman-owl HIGH	persistent-mem MED	deploy-mgr MED	browser-use LOW
Browser-Use	opencode-owl HIGH	openhuman-owl HIGH	deploy-mgr MED	persistent-mem MED	mcp-builder MED	combined-bill LOW-MED	api-gateway LOW
Deployment Mgr	combined-bill HIGH	api-gateway HIGH	openrelay HIGH	opencode-owl MED	openhuman-owl MED	persistent-mem MED	browser-use MED
Persistent Memory	opencode-owl HIGH	openhuman-owl HIGH	combined-bill HIGH	api-gateway HIGH	openrelay MED	deploy-mgr MED	browser-use MED
MCP Builder	combined-bill HIGH	api-gateway MED	persistent-mem MED	openhuman-owl LOW-MED	openrelay LOW-MED	deploy-mgr LOW-MED	browser-use LOW

8. Incompatibility Matrix (Not Compatible Types)

Skill	Incompatible With	Severity	Reason
Opencode Owl	Hardcoded API keys	HIGH	Keys bypass proxy entirely
Opencode Owl	VPN-level proxies	MEDIUM	Double-proxy TLS failures
Openhuman Owl	Service accounts without refresh tokens	HIGH	No refresh tokens = no auto re-auth
Openhuman Owl	VPN proxies (OAuth)	MEDIUM	OAuth2 PKCE redirect fails through VPN
Combined Billing	Direct API keys (no proxy)	HIGH	No billing events to aggregate
Combined Billing	Another combined billing instance	HIGH	Double-counting of events
OpenRelay Go	TLS certificate pinning	HIGH	Pinned apps reject TLS interception
OpenRelay Go	VPN tunnels to AI providers	MEDIUM	VPN bypasses network interception
API Gateway	Another API gateway	HIGH	Routing loops between two gateways
API Gateway	Hardcoded provider URLs	HIGH	Application bypasses gateway
Browser-Use	CAPTCHA-protected pages	HIGH	Cannot solve CAPTCHAs autonomously
Browser-Use	Desktop (non-browser) apps	HIGH	Only automates web browsers
Deployment Mgr	Manual deployment processes	HIGH	Conflicting deployment paradigms
Deployment Mgr	Another deployment manager	HIGH	Two systems fighting for control
Persistent Memory	Large binary data	HIGH	Causes memory pressure; use object storage
Persistent Memory	High-throughput event streams	MEDIUM	Use a message queue instead
MCP Builder	Non-MCP agent frameworks	MEDIUM	MCP servers only work with MCP clients
MCP Builder	Real-time streaming protocols	MEDIUM	Request-response model; no streaming

9. Stackability Analysis

All nine skills are stackable. None is inherently unstackable — each can be combined with others to form a more capable system. However, the stacking principles differ by layer: client-layer skills (owl installs) stack by coexisting on the same host with separate env vars; service-layer skills (billing, gateway, relay) stack by forming a processing pipeline (relay intercepts, gateway routes, billing tracks); infrastructure-layer skills (deployment, persistent memory) stack as the foundation that everything else depends on; and integration-layer skills (browser-use, MCP builder) stack as glue that automates workflows and exposes capabilities.

Skill	Stackable?	Stacking Principle	Key Stack Position
Opencode Owl	YES	Outermost application layer: configures the client	Entry point (client side)
Openhuman Owl	YES	Coexists with Opencode owl via separate env vars	Entry point (client side)
Combined Billing	YES	Top of the billing stack: consumes events from all proxies	Aggregation layer
OpenRelay Go	YES (caveats)	Network layer: intercepts before application layer	Interception layer
API Gateway	YES	Hub/spine: all other skills connect to or through it	Routing hub

Skill	Stackable?	Stacking Principle	Key Stack Position
Browser-Use	YES	Glue skill: automates interactive steps for other skills	Automation glue
Deployment Mgr	YES	Floor/infrastructure: everything else builds on top	Infrastructure base
Persistent Memory	YES	Substrate/foundation: provides state for all skills	State foundation
MCP Builder	YES	API layer: makes other skills' capabilities agent-accessible	Agent interface

10. Best Combination Synergies

Combination	Synergy Level	Why It Works
Persistent Memory + ALL skills	CRITICAL	Every skill benefits from state persistence; PM is the universal substrate
Combined Billing + API Gateway	MAXIMUM	Billing tracks costs, gateway routes requests = full-stack AI API management
Opencode Owl + Openhuman Owl	MAXIMUM	Both AI coding and human-AI workflow billing covered on same host
API Gateway + OpenRelay Go	HIGH	Network interception + application routing = defense in depth
Opencode Owl + API Gateway	HIGH	Owl configures client to use gateway; gateway routes to provider
Openhuman Owl + Browser-Use	HIGH	Browser-use automates OAuth2 PKCE consent during install
Combined Billing + OpenRelay Go	HIGH	OpenRelay intercepts, Combined Billing aggregates = complete visibility
Deployment Mgr + All services	HIGH	Automated deployment + monitoring of billing, gateway, and relay services
MCP Builder + Combined Billing	HIGH	Billing data becomes AI-agent-queryable via MCP tools
Persistent Memory + Openhuman Owl	HIGH	OAuth tokens survive watchdog restarts, auto re-auth works

11. Worst Combination Conflicts

Combination	Conflict Level	Why It Fails
OpenRelay Go + TLS pinning apps	FATAL	Pinned apps reject OpenRelay's TLS interception entirely
Combined Billing + Another billing instance	CRITICAL	Double-counting produces inaccurate totals
API Gateway + Another API Gateway	CRITICAL	Routing loops between two gateways
Deployment Mgr + Manual deployments	CRITICAL	Conflicting paradigms cause inconsistent state
Openhuman Owl + VPN (OAuth)	SEVERE	OAuth2 PKCE redirect fails through VPN interception
Browser-Use + CAPTCHA-heavy sites	SEVERE	Constant human intervention defeats automation purpose
Persistent Memory + Large binaries	SEVERE	Memory pressure; use object storage instead
Opencode Owl + Hardcoded API keys	HIGH	Keys bypass the proxy entirely, defeating its purpose
OpenRelay Go + VPN tunnels	HIGH	VPN bypasses OpenRelay's network interception
MCP Builder + Non-MCP frameworks	HIGH	MCP servers cannot communicate with non-compliant clients

12. Integration Architecture — Layered Stack Diagram

The nine skills form a layered architecture with four distinct layers. The diagram below shows how data flows from client applications through the infrastructure stack and back.

LAYER 4: AGENT INTERFACE	MCP Builder (billing tools, routing tools, quota tools)
LAYER 3: SERVICES	API Gateway (routing, rate limiting, caching) + Combined Billing (aggregation, quotas, anomalies) + OpenRelay Go (network interception, failover)
LAYER 2: CLIENT CONFIG	Opencode Owl Install (OpenCode CLI proxy config) + Openhuman Owl Install (OpenHuman runtime proxy config + OAuth2 PKCE)
LAYER 1: INFRASTRUCTURE	Persistent Memory (tokens, quotas, policies, state) + Deployment Manager (build, deploy, monitor, rollback) + Browser-Use (OAuth automation, UI testing)

Data Flow: Request Lifecycle

Step	Component	Action	Skill Responsible
1	Client (OpenCode/OpenHuman)	User sends AI request	Owl Install Proxy (configured endpoint)
2	Network Intercept	OpenRelay intercepts outbound HTTPS to AI endpoint	OpenRelay Go
3	API Gateway	Routes to optimal provider based on policy	API Gateway Skill
4	Upstream Provider	Processes request, returns response	External (Claude, GPT-4, etc.)
5	Billing Aggregation	Ingests billing event, updates quota, checks anomalies	Combined Proxy Billing
6	State Persistence	Persists tokens, quotas, routing policies	Persistent Memory
7	Agent Interface	AI agent queries spending/quota via MCP tools	MCP Builder

13. File Inventory & Local Storage Map

All skill definitions and scripts have been written to local storage under `/home/z/my-project/skills/`. Each skill directory contains a `profile.md` (11-dimension analysis) and a main script (`install.sh`, `billing-server.py`, `relay.go`, `gateway.py`, `pmem.py`, or `billing-mcp-server.ts`). Existing `SKILL.md` files (`browser-use`, `deployment-manager`, `mcp-builder`) have been preserved.

Skill Directory	Files Written	Script Type
<code>opencode-owl-install-proxy/</code>	<code>profile.md</code> , <code>install.sh</code>	Bash (bootstrap + OS detection + validation)
<code>openhuman-owl-install-proxy/</code>	<code>profile.md</code> , <code>install.sh</code>	Bash (bootstrap + OAuth2 PKCE + watchdog daemon)
<code>combined-proxy-billing/</code>	<code>profile.md</code> , <code>billing-server.py</code>	Python HTTP server (SQLite + events + quotas + anomalies)
<code>openrelay-go/</code>	<code>profile.md</code> , <code>relay.go</code>	Go daemon (auto-discovery + failover + warmup + registry)
<code>api-gateway-skill/</code>	<code>profile.md</code> , <code>gateway.py</code>	Python HTTP server (routing + rate limiting + caching)
<code>browser-use/</code>	<code>profile.md</code> (+ existing <code>SKILL.md</code>)	Natural-language browser automation
<code>deployment-manager/</code>	<code>profile.md</code> (+ existing <code>SKILL.md</code>)	Full-lifecycle deployment orchestration
<code>persistent-memory/</code>	<code>profile.md</code> , <code>pmem.py</code>	Python HTTP server (KV store + TTL + OCC + snapshots)
<code>mcp-builder/</code>	<code>profile.md</code> , <code>billing-mcp-server.ts</code> (+ existing <code>SKILL.md</code>)	TypeScript MCP server (4 billing tools)